Project Progress Report

- First, I set up the project structure, created the required packages, configured the database connection pool, and built the core entities and enums.
- Next, I developed an **Audit Log System** (consisting of an Audit Annotation, Interceptor, and Logger Bean) to automatically log system activities:

    1. **Audit.java (Annotation):** Created a new @Audit annotation. When placed on any business method or class, the system recognizes that it needs to be logged. With @Inherited included, sub-classes automatically inherit this logging behavior.
    2. **CurrentUserContext.java:** A Request-scoped bean that temporarily captures and holds the User ID and Username of the currently logged-in user during an HTTP request. This is used to track who placed an order or modified data in the audit logs.
    3. **AuditInterceptor.java (Interceptor):** Handles multiple tasks when a business method runs. It notes the start time, calculates the total execution time upon success, determines the audit action based on the method name, and delegates saving to the AuditLoggerBean. Using @Priority(Interceptor.Priority.APPLICATION) avoids extra configuration in beans.xml.
    4. **AuditLoggerBean.java:** Uses @TransactionAttribute(TransactionAttributeType.REQUIRES_NEW) so that even if a business method fails and causes a database rollback, the audit log runs in a separate transaction and is permanently saved in the database.

Vendor Module & CRUD Operations

- Configured database constraints for the Vendor entity (Unique constraint on TRADELICENSENO, NOT NULL on NAME, CONTACTEMAIL, and COUNTRY).
- Implemented and tested all CRUD operations (POST, GET, PUT, and DELETE) for Vendors using integer IDs via VendorPerformanceBean and VendorResource.
- Configured update and delete data changes and transactions to be automatically logged through the Audit Log system.

Security and RBAC (Role-Based Access Control)

- Secured the VendorResource class using @RolesAllowed({"ADMIN", "VENDOR"}).

- Successfully tested and verified unauthorized requests (401 Unauthorized) and forbidden access attempts (403 Forbidden) via Postman using the JWT Authentication Filter.

Warehouse Management

- Completed CRUD operations using WarehouseManagementBean and WarehouseResource to add new warehouses (POST /warehouses/add) and view their details under a multi-warehouse approach.
- Enforced security by restricting warehouse creation exclusively to Admins using @RolesAllowed({"ADMIN"}).
- Implemented inventory operations (receiveStock and dispatchStock). Built WarehouseInventoryBean and InventoryResource to deduct stock from specific warehouse IDs, and handled issues like InsufficientStockException when stock is low.

Purchase Order

- Created the Purchase Order (PO) management module, including the PurchaseOrder entity and OrderStatus enum (PENDING, APPROVED, SHIPPED, COMPLETED, etc.) to handle orders per vendor.
- Linked the OrderItem entity to view item details (Items, Quantity, Unit Price) inside orders, resolving infinite loop (recursive reference) issues during JSON serialization using the @JsonbTransient annotation.
- Developed the business logic in PurchaseOrderBean (service layer) to create new purchase orders (createPurchaseOrder), find them by ID, and update statuses using @Stateless EJBs and @Audit logging.
- Created POST and GET APIs in PurchaseOrderResource (REST controller) and secured them with @RolesAllowed({"ADMIN", "PROCUREMENT_OFFICER"}) to ensure only authorized personnel can create and view orders.

Shipment Management

- Created the Shipment entity (with Carrier, PurchaseOrder, origin, destination, dispatchDate, eta, status, and events) and ShipmentEvent entity (with EventType, eventTime, and notes).
- Resolved infinite recursion (circular reference) errors during JSON serialization by properly placing the @JsonbTransient annotation on lists and foreign key fields across PurchaseOrder, Shipment, and ShipmentEvent entities.

- Implemented shipment creation and tracking functionalities via EJBs using ShipmentManagementBean (service layer).
- Built the necessary APIs using ShipmentResource (REST controller) and successfully verified all progress by receiving 200 OK and 201 Created responses in Postman.